

[← Go Back](#)

Hardware

Nano 33 IoT



Tutorials

[Controlling a LED Through Bluetooth® with Nano 33 IoT](#)[Connecting Two Nano 33 IoT Boards Through I2C](#)[Sending an Email Through IFTTT with Nano 33 IoT](#)[Accessing Accelerometer Data on Nano 33 IoT](#)[Accessing Gyroscope Data on Nano 33 IoT](#)[Nano 33 IoT Alarm with the Arduino Cloud](#)[Connecting Two Nano 33 IoT Boards Through UART](#)[Connecting the Nano 33 IoT to a Wi-Fi Network](#)

Home / Hardware / Nano 33 IoT / [Connecting the Nano 33 IoT to a Wi-Fi Network](#)

Connecting the Nano 33 IoT to a Wi-Fi Network

Learn how to connect the Nano 33 IoT board to a Wi-Fi network with the WiFiNINA library.

Author · Fabricio Troya · Last revision · 07/17/2024

This tutorial will connect an Arduino Nano 33 IoT to the Wi-Fi network, by using the **NINA-W102** module embedded on the board. To do so, we will use the module to create an application that will connect to the Wi-Fi network, measure its strength signal, and make a LED blink based on the strength of the signal.

Goals

The goals of this project are:

- Understand what the NINA-W102 module is.

- Use the WiFiNINA library.

- Measure the signal strength of a Wi-Fi network.

- Convert the signal strength values in blinking speed of the LED.

ON THIS PAGE

Goals

[Hardware & Software Needed](#)[The NINA-W102 Module](#) +[Creating the Program](#)[Testing It Out](#) +[Conclusion](#)[Help](#)

Hardware & Software Needed

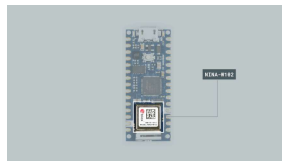
This project uses no external sensors or components.

In this tutorial we will use the Arduino Cloud Editor to program the board.

The NINA-W102 Module

The NINA-W102 module is a stand-alone multiradio MCU modules module that integrate a powerful microcontroller (MCU) and a radio for wireless communication. The radio provides support for Wi-Fi 802.11 b/g/n in the 2.4 Hz ISM band and Bluetooth® v4.2 (Bluetooth® BR/EDR and Bluetooth® low energy) communications.

If you want to learn more about Wi-Fi and how it works, you can the check [this link](#).



The NINA-W102 module.

The NINA-W102 includes the wireless MCU, Flash memory, crystal, and components for matching, filtering, antenna and decoupling, making it a very compact stand-alone multiradio module. The module can be used to design solutions with top grade security, thanks to integrated cryptographic hardware

The Library

The WiFinINA library allows us to use the Arduino Nano 33 IoT NINA-W102 module without having to go into complicated programming. The library supports **WEP**, **WPA2 Personal** and **WPA2 Enterprise** encryptions and it can serve as either a **server** accepting incoming connections or a **client** making outgoing ones. Some of the most common applications for the module are the following:

Internet of Things

Wi-fi networks

**Bluetooth® and
Bluetooth® low energy
applications**

Telematics

**Access to laptops, mobile
phones, and similar
consumer devices**

**Medical and industrial
networking**

**Home/building
automation**

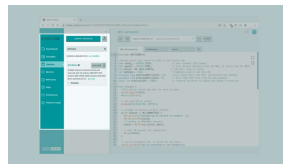
If you want to read more about the NINA-W102 module see [here](#).

In this tutorial we will read the information of a Wi-Fi network as the SSID, the IP of the board and the strength signal. With this information we will be able to make the LED_BUILTIN of the board blink faster or slower based on the signal strength of the network.

Creating the

1. Setting up

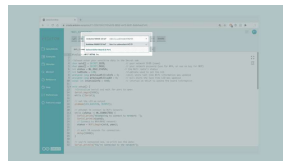
Let's start by opening the [Arduino Cloud Editor](#) and creating a new sketch, this can be named "WiFi_Connection". Then, navigate to the **Libraries** tab, search for the **WiFinINA** library and click on the **Include** button.



Finding the library in the Cloud Editor.

2. Connecting the board

Next, connect the Arduino Nano 33 IoT to the computer and make sure that the Cloud Editor recognizes it. If so, the board and port should appear as shown in the image below. If they don't appear, follow the [instructions](#) to install the plugin that will allow the Editor to recognize your board.



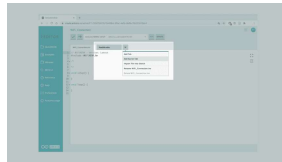
Selecting the board.

3. Connecting to Wi-Fi

After including the library, we can begin by building the rest of the code.

Let's start by creating a header file to store our credential for the Wi-Fi network which we will

accidentally store our credentials in a code we may share on the internet. To do this, we need to click the downward arrow next to our sketch tab, and click "**Add Secret Tab**". This will create a tab called **Secret**.

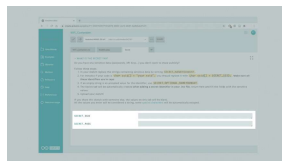


Adding a secret tab.

We now need to head back to the original sketch file, and enter the following code:

```
1 //please enter your sen  
2 char ssid[] = SECRET_SS  
3 char pass[] = SECRET_PA
```

This will automatically create two fields in the **Secret** tab. If we go back to this tab, we can enter our credentials there.



Entering the
credentials for your
network.

Now, back to the sketch, we can continue coding. We need to initialize five more variables to store some information as the Wi-Fi radio's status, the state of the

```
1 int status = WL_IDLE_ST
2 int ledState = LOW;
3 unsigned long previousM
4 unsigned long previousM
5 const int intervalInfo
```

In the `setup()` we start serial communication at 9600 bauds, followed by `while(!Serial);`, which basically means that unless we open the Serial Monitor the program will not run. We then use a `pinMode()` to set the LED_BUILTIN pin as an `OUTPUT`.

```
1 void setup() {
2   //Initialize serial a
3   Serial.begin(9600);
4   while (!Serial);
5
6   // set the LED as out
7   pinMode(LED_BUILTIN,
```

Then, we create a `while()` loop that checks if we are connected to Wi-Fi, to begin connecting to it.

We then use

```
status = WiFi.begin(ssid,
pass);
```

to start connecting to Wi-Fi, and a delay of 10 seconds to give it enough time to connect.

```
1 // attempt to connect
2 while (status != WL_CO
3     Serial.print("Attemp
4     Serial.println(ssid)
5     // Connect to WPA/WP
6     status = WiFi.begin(
7
8     // wait 10 seconds f
9     delay(10000);
10 }
```

At the end of the `setup()` we use two `Serial.println()` functions to print out in the serial monitor that we are now connected to the network and to separate the block of data.

```
1 // you're connected now
2 Serial.println("You're
3 Serial.println("-----
```

In the `loop()` we start initializing a new variable that will store the time since the sketch start running.

```
1 void loop() {
2     unsigned long current
```

Then, we will use an `if()` statement to check if the time after the last update is bigger than the interval we set before.

```
1 // check if the time af
```

Inside the `if()` statement, we start setting the

`previousMillisInfo` variable equal to the `currentMillisInfo` and then we print out three different types of information:

Board's IP address using the `WiFi.localIP()` function.

Name of network connected to using the `WiFi.SSID()` function.

Signal strength using the `WiFi.RSSI()` function.

```
1  previousMillisInfo = c
2
3      Serial.println("Bo
4      // print your boar
5      IPAddress ip = WiF
6      Serial.print("IP A
7      Serial.println(ip)
8
9      // print your netw
10     Serial.println();
11     Serial.println("Ne
12     Serial.print("SSID
13     Serial.println(WiF
14
15     // print the recei
16     long rssi = WiFi.R
17     Serial.print("sign
18     Serial.println(rss
19     Serial.println("--
20 }
```

After this, we need to initialize another two variables, one to keep track of the time for the LED and another one to convert the signal strength measured for the `WiFi.RSSI()` function to a time interval.


```
1 unsigned long currentMi
2
3 // measure the signal s
4 int intervalLED = WiFi.
```

Lastly, we create another `if()` statement to check if the time after the last blink is bigger than the interval, in the same way we did before. Inside the `if()` statement we add a new `else if()` statement to turn the LED on if it was off and vice-versa, and a `digitalWrite()` function to set the state of the LED.

```
1 // check if the time a
2 if (currentMillisLED
3     previousMillisLED
4
5     // if the LED is o
6 if (ledState == L0
7     ledState = HIGH;
8 } else {
9     ledState = LOW;
10 }
11
12 // set the LED wit
13 digitalWrite(LED_B
14 }
15 }
```

4. Complete code

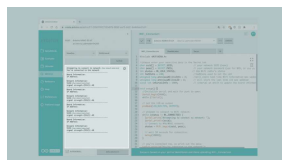
If you choose to skip the code building section, the complete code can be found below:



```
1  #include <WiFiNINA.h>
2
3  //please enter your s
4  char ssid[] = SECRET_
5  char pass[] = SECRET_
6  int status = WL_IDLE_
7  int ledState = LOW;
8  unsigned long previou
9  unsigned long previou
10 const int intervalInf
11
12 void setup() {
13     //Initialize serial
14     Serial.begin(9600);
15     while (!Serial);
16
17     // set the LED as c
18     pinMode(LED_BUILTIN
19
20     // attempt to conne
21     while (status != WL
22         Serial.print("Att
23         Serial.println(ss
24         // Connect to WPA
25         status = WiFi.beg
26
27         // wait 10 second
28         delay(10000);
```

Testing It Out

After you have successfully verified and uploaded the sketch to the board, it's time to put it to the test. Don't forget to enter the credentials of the network in the **Secret** tab. Once the board it's connected to the network we can see the information in the Serial Monitor.



Information about

Move the board around and you will see how the signal strength changes as well as the blinking speed of the board's LED. The signal strength is measure in **dBm** (decibel-milliwatts) which means that the lower the number is, the stronger the connection, and the faster the LED will blink.

Troubleshoot

Sometimes errors occur, if the code is not working there are some common issues we can troubleshoot:

- Missing a bracket or a semicolon.

- Arduino board connected to the wrong port.

- Missing the SSID and PASS in the Secret tab.
Remember, it is a case sensitive.

Conclusion

In this tutorial, we have learned how to simply connect to a Wi-Fi network by using the credentials in the code. We also learned how to obtain specific information regarding our connection, such as signal strength, IP address and name of our network, and then use that information to make a LED blink faster or slower.

docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page [here](#).

Ask the Arduino Forum Discover Arduino Discord

under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

